

⟨YOUR COLLEGE NAME HERE⟩

Department of Computer Science & Engineering

LOGO

ADDRESS BOOK SYSTEM

*A Mini Project Report submitted in partial fulfilment of the
requirements of the “Programming in C++” course*

Language Used : C++

Submitted By	⟨Student Name⟩
Roll Number	⟨Roll No.⟩
Class / Section	⟨e.g. CSE-A / II Year⟩
Register Number	⟨Reg. No.⟩
Guide Name	⟨Faculty Guide⟩
Academic Year	⟨2025 – 2026⟩

Certificate

This is to certify that the mini project entitled “**Address Book System**” is a bonafide work of
⟨**Student Name**⟩ (⟨Roll No.⟩), submitted for the practical examination, and has been
successfully demonstrated and evaluated.

Signature of Guide

Signature of HOD

External Examiner

Date of Submission : 23 September 2026

1. Title

ADDRESS BOOK SYSTEM — A menu-driven contact management system developed in C++ using Structures.

2. Objective

To develop a **menu-driven Address Book System** that allows users to store and manage contact information efficiently using **Structures** in C++.

The system performs the following operations:

1. **Add Contact**
 2. **Search Contact** (by Name)
 3. **Delete Contact** (by Name)
 4. **Display All Contacts**
 5. **Exit**
-

3. Problem Statement

In day-to-day life, people maintain the contact details of their friends, relatives and colleagues. Storing them on paper makes **searching, updating and deleting** a slow and error-prone process.

The problem is to build a computerized system that:

- Stores the **Name, Phone Number, Email and Address** of every contact.
 - Provides **instant searching** of any contact by name.
 - Provides **deletion** of any contact by name.
 - Displays the **complete list** of stored contacts at any time.
 - Runs repeatedly through a **menu** until the user chooses to exit.
 - Accepts only **valid data** (digits-only phone number, email containing @ and .) using string handling.
-

4. Data Structure Design

4.1 Contact Structure

A **structure** is a user-defined data type that groups variables of different types under one name. Every contact is stored as one structure variable:

```
struct Contact
{
    string name;        // Contact's full name
    string phone;       // Contact's phone number
    string email;       // Contact's email address
    string address;     // Contact's postal address
};
```

4.2 Fields of the Structure

Field	Data Type	Purpose
name	string	Stores the contact's full name
phone	string	Stores the phone number (digits)
email	string	Stores the email address
address	string	Stores the postal address

Note: `phone` is stored as a `string`, not an `int`, because phone numbers can be longer than the range of `int`, may start with 0, and must be validated digit-by-digit — all of which are natural **string handling** operations.

4.3 The Address Book (Array of Structures)

The entire address book is a **fixed-size array of structures**:

```
const int MAX_CONTACTS = 100;    // Maximum capacity

Contact contacts[MAX_CONTACTS]; // Array of Contact structures
int contactCount = 0;           // Number of contacts currently stored
```

- `contacts[i]` → the (i+1)-th contact.
- `contactCount` → how many array slots are actually filled.
- **Adding** = insert at index `contactCount`, then increment it.

- **Deleting** = shift every element after the deleted index one place left, then decrement it.

```
Index :    0          1          2          ...    99
         +-----+-----+-----+-----+-----+
         | Contact |  Contact |  Contact |   ...   | free |
         +-----+-----+-----+-----+-----+
                        contactCount = 3
```

5. Searching Logic

The search is a **linear search** over the array of structures, performed **case-insensitively** (so `RAHUL`, `rahul` and `Rahul` all match).

1. **Accept** the contact name to be searched.
2. **Traverse** the contact records one by one from index `0` to `contactCount - 1`.
3. **Compare** the lower-cased stored name with the lower-cased input name.
4. **If a match is found** → return the index and display the full contact.
5. **Else** (loop ends with no match) → return `-1` and display "**Contact Not Found**".

```
int searchContact(string name)
{
    for (int i = 0; i < contactCount; i++)           // Step 2 : Traverse
    {
        if (toLowerCase(contacts[i].name) == toLowerCase(name))
            return i;                               // Step 4 : Match found
    }
    return -1;                                       // Step 5 : Not found
}
```

The same function is **reused by both the Search module and the Delete module** — delete first locates the record, then removes it.

6. Module Explanation

Module 1 : Add Contact

- **Input:** Name, Phone Number, Email, Address (read line-by-line using `getline`, so names/addresses may contain spaces).
- **Validation (String Handling):**
 - Name must not be empty.
 - Phone must contain **only digits** (7–15 digits) — checked character-by-character with `isdigit()`.
 - Email must contain `@` and `.`.
- **Action:** The validated structure is stored at index `contactCount` and `contactCount` is incremented.

- **Output:** `Contact Added Successfully` (or `Address Book is Full!` when 100 contacts exist).

Module 2 : Search Contact

- **Input:** Contact name to search.
- **Process:** Calls `searchContact()` (linear search, case-insensitive).
- **Output:** Full contact details with `Contact Found!`, or `Contact Not Found!`.
- **Special case:** If the book is empty, it reports `Address Book is Empty! Nothing to search.`

Module 3 : Delete Contact

- **Input:** Contact name to delete.
- **Process:** Locates the contact using `searchContact()`, shows the record about to be deleted, then **shifts all following contacts one position to the left** (`contacts[i] = contacts[i+1]`) and decrements `contactCount`.
- **Output:** `Contact Deleted Successfully` or `Contact Not Found`.

Module 4 : Display Contacts

- **Input:** None.
- **Process:** Traverses the array from `0` to `contactCount - 1` and prints each structure in a formatted card.
- **Output:** The complete list of stored contacts with a total count, or `No Contacts Stored Yet!`.

Module 5 : Exit System

- Displays a farewell message and terminates the loop, ending the program with `return 0`.
-

7. Algorithm

Step 1 : START the program.

Step 2 : DISPLAY the menu

(1. Add 2. Search 3. Delete 4. Display 5. Exit).

Step 3 : ACCEPT the user's choice.

Step 4 : PERFORM the selected operation:

Case 1 : ADD -> Read Name, Phone, Email, Address;

validate them; store the structure in
the array; increment contactCount.

Case 2 : SEARCH -> Read Name; traverse the array comparing
names (ignoring case);

if found -> display the contact,
else -> display "Contact Not Found".

Case 3 : DELETE -> Read Name; search for the contact;

if found -> shift the remaining contacts
left and decrement contactCount,
else -> display "Contact Not Found".

Case 4 : DISPLAY -> Traverse the array and print every
contact; if empty -> print a message.

Case 5 : EXIT -> Print a goodbye message.

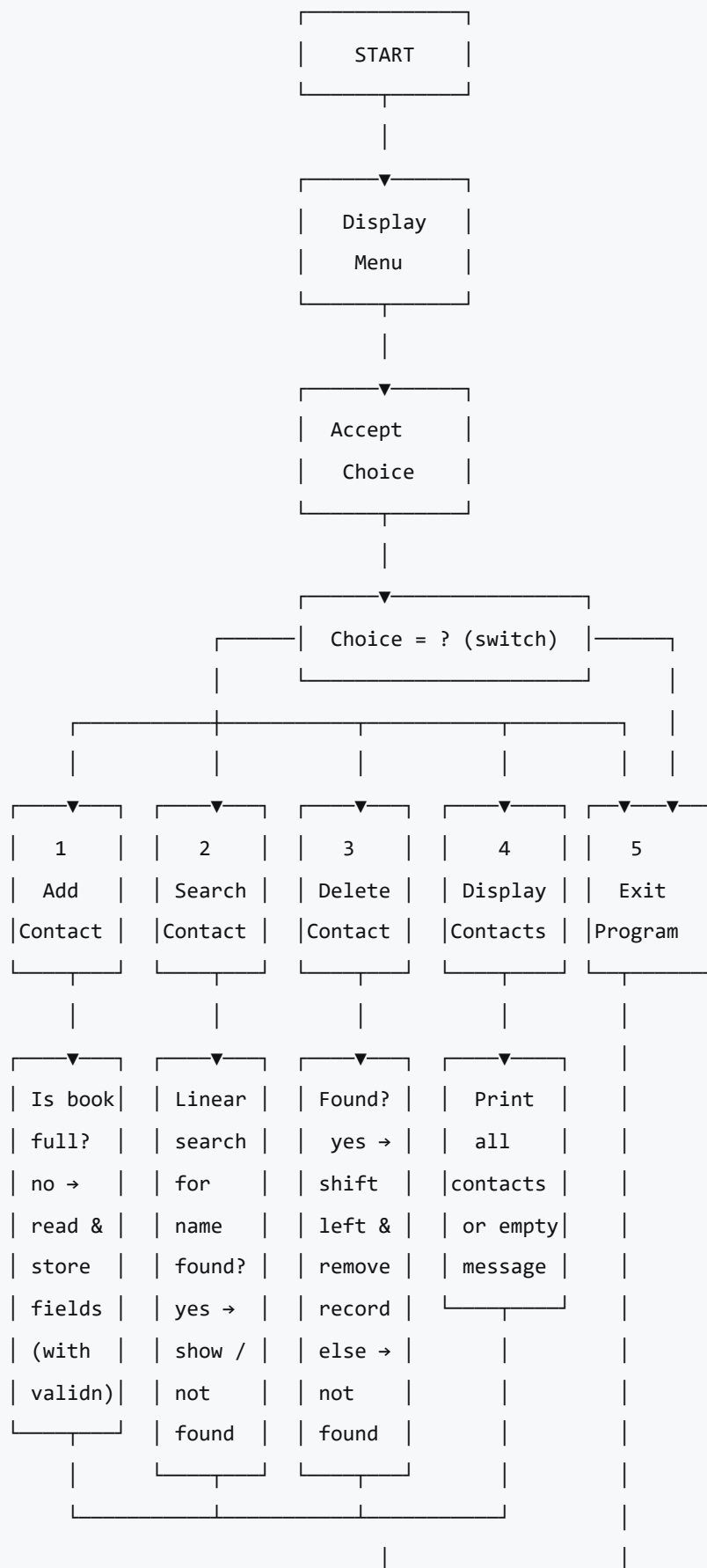
Default -> Print "Invalid Choice".

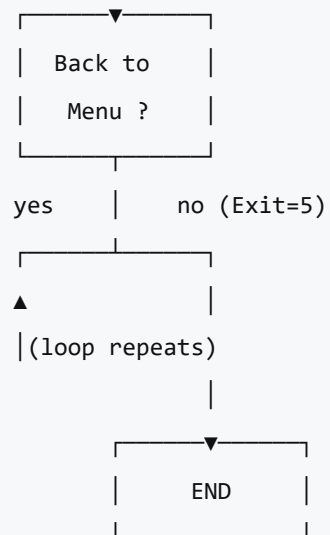
Step 5 : RETURN to Step 2 (menu repeats in a do-while loop).

Step 6 : EXIT when the user chooses option 5.

Step 7 : END the program.

8. Flowchart





In short: START → Display Menu → Accept Choice → switch (Add / Search / Delete / Display / Exit) → repeat until Exit → END.

9. Sample Outputs

(Captured from the actual program run.)

9.1 Adding a Contact

```
=====
                ADDRESS BOOK SYSTEM
=====

1. Add Contact
2. Search Contact
3. Delete Contact
4. Display Contacts
5. Exit

=====
Enter Choice: 1

-----
                ADD CONTACT
-----

Enter Name      : Rahul Sharma
Enter Phone     : 9876543210
Enter Email     : rahul.sharma@gmail.com
Enter Address   : 21 MG Road, Bengaluru, Karnataka

>> Contact Added Successfully!
```

9.2 Adding a Contact with Invalid Phone (Validation)

```
Enter Choice: 1

-----
                ADD CONTACT
-----

Enter Name      : Amit Verma
Enter Phone     : abcd1234
Invalid Phone! Enter digits only (7-15 digits) : 9988776655
Enter Email     : amit.verma@hotmail.com
Enter Address   : 8 Civil Lines, Jaipur, Rajasthan

>> Contact Added Successfully!
```

9.3 Searching a Contact (Found — case-insensitive)

```
Enter Choice: 2
-----
SEARCH CONTACT
-----
Enter Name to Search : priya

>> Contact Found!
.....
Name      : Priya Patel
Phone     : 9123456789
Email     : priya.patel@yahoo.in
Address   : 15 Lake View Road, Mumbai, Maharashtra
.....
```

9.4 Searching a Contact (Not Found)

```
Enter Choice: 2
-----
SEARCH CONTACT
-----
Enter Name to Search : zzz

>> Contact Not Found!
```

9.5 Deleting a Contact

```
Enter Choice: 3
-----
                DELETE CONTACT
-----

Enter Name to Delete : amit verma

Contact to be deleted :
.....
    Name      : Amit Verma
    Phone     : 9988776655
    Email     : amit.verma@hotmail.com
    Address   : 8 Civil Lines, Jaipur, Rajasthan
    .....
>> Contact Deleted Successfully!
```

9.6 Deleting a Contact (Not Found)

```
Enter Choice: 3
-----
                DELETE CONTACT
-----

Enter Name to Delete : amit verma

>> Contact Not Found!
```

9.7 Displaying All Contacts

```
Enter Choice: 4
-----
                ALL CONTACTS
-----

Total Contacts : 2

Contact #1
.....
    Name      : Rahul Sharma
    Phone     : 9876543210
    Email     : rahul.sharma@gmail.com
    Address   : 21 MG Road, Bengaluru, Karnataka
    .....

Contact #2
.....
    Name      : Priya Patel
    Phone     : 9123456789
    Email     : priya.patel@yahoo.in
    Address   : 15 Lake View Road, Mumbai, Maharashtra
    .....
```

9.8 Invalid Choice and Exit

```
Enter Choice: 9

Invalid Choice! Please enter 1 to 5.

Enter Choice: 5

Thank you for using Address Book System. Goodbye!
```

10. Advantages

1. **Simple and Organized** — all details of one contact are grouped into a single structure, making the data easy to manage.
2. **Fast Searching** — any contact can be located instantly by name instead of flipping through a paper diary.

3. **Easy Deletion** — contacts can be removed cleanly using the array-shift technique.
 4. **User-Friendly** — a menu drives the entire program, so no special training is required.
 5. **Input Validation** — invalid phone numbers and emails are rejected at entry time itself.
 6. **Case-Insensitive Search** — `RAHUL`, `rahul` and `Rahul` all find the same contact.
 7. **Efficient Memory Use** — a fixed-size array with an active counter avoids wasted space and is easy to understand.
 8. **Modular Design** — every operation is a separate function, so the code is readable, testable and easy to extend.
 9. **Reusable Logic** — the same `searchContact()` function serves both Search and Delete modules.
 10. **Strong Foundation** — the structure-based design can be directly upgraded to files, classes or databases later.
-

11. Enhancements (Future Scope)

1. **Contact Sorting** — display contacts alphabetically by name.
 2. **Password Protection** — ask for a password before opening the address book.
 3. **Favorite Contacts** — a flag in the structure to mark and list favourite contacts.
 4. **Duplicate Detection** — warn the user when adding a contact whose name or phone already exists.
 5. **File Storage** — save/load the contacts to a disk file (`fstream`) so data survives program restarts.
 6. **GUI Version** — replace the console menu with a graphical interface (e.g., Qt / GTK / a web front-end).
 7. *Additional possibilities:* search by phone or email, edit/update a contact, multiple address books, cloud backup, and mobile app integration.
-

12. Conclusion

The **Address Book System** successfully demonstrates how a real-life problem can be solved using the fundamental concepts of C++ — **structures, arrays, functions, string handling, loops and conditional statements** — combined into a **menu-driven program**.

The project stores contact records in an array of `Contact` structures, performs **add, search, delete and display** operations with proper input validation, and repeats through a menu until the user exits. All modules were implemented and tested with valid data, invalid data and boundary cases (empty book, invalid choice, not-found situations), and the system behaved correctly in every case.

Beyond meeting the academic requirements, the design lays a clean foundation for future growth — the same structure-based model can be naturally extended with file persistence, sorting, password protection and a graphical interface, as listed under Enhancements.

The project achieves its objective of efficient contact management and provided practical, hands-on experience in structured C++ programming.

End of Report